UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026

Applied Parallel Programming

# Lecture 16: Advanced Optimizations for Projects

Hrishi Shah

03/24/2025

# Course Logistics

Start working on Project Milestone 2 if you have not already!

Lab 6 (List Reduction) is due this week.

Profiling on Delta can take a while! Do not wait to complete the milestone. If your profiling job fails and you have the proper flags, then it may be a system issue, you should re-submit the job.

Be sure to come to lecture this Thursday, interesting dive into a major part of NVIDIA GPUs, and a part of both projects.

Recorded profile lecture 13, do not skip just because it’s recorded!

# Milestone 2 Info (GPT)

README_M2.md has all the information on this Milestone

README.md has also been slightly updated to reflect the point breakdowns

Due **April 10th** (without 3-day extension).

GPT has code + report (Gradescope) + demo components

CNN has code + report (Prairielearn) components

# Matrix Multiplication

Core operation in scientific computing, graphics processing, data analysis, and deep learning (can be up to 99% of FLOPs in ML models!)

General (dense) matrix multiplication is both compute and memory intensive

Many hardware components and software libraries are specifically built for matmul.

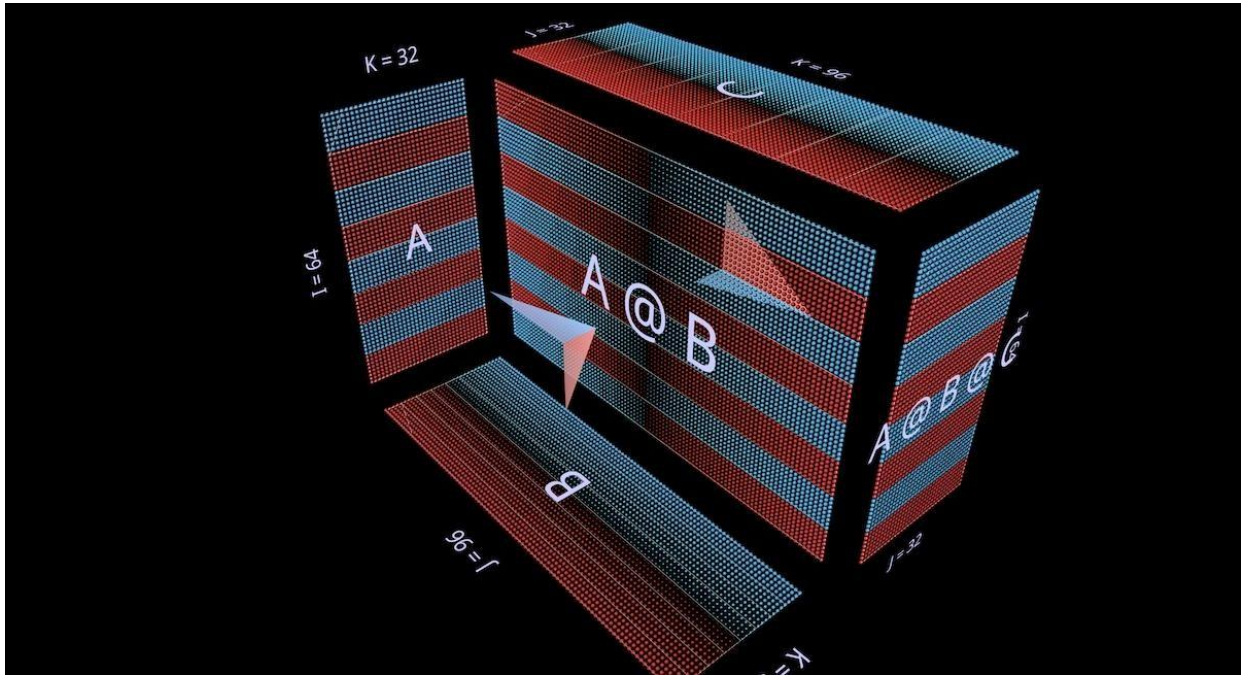



Image Source: PyTorch

# Where We Left Off: Basic Shared Memory Tiling

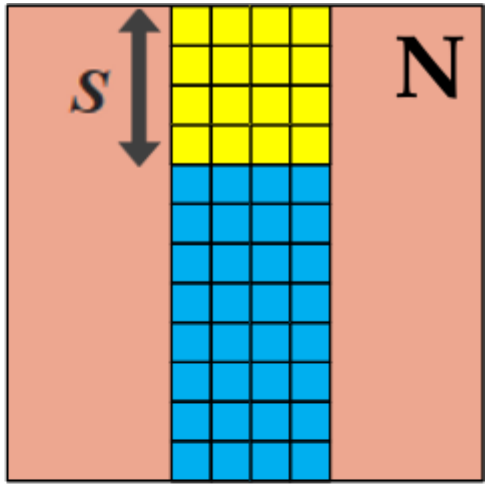
Review:

Basic Shared Memory Tiled Matrix Multiplication (Lab 3)

TxU block of threads computes a TxU block of output values

Loads tiles of size TxS from input M and size SxU from input N

In your lab and previous lectures, T = U = S (usually, 16 or 32)



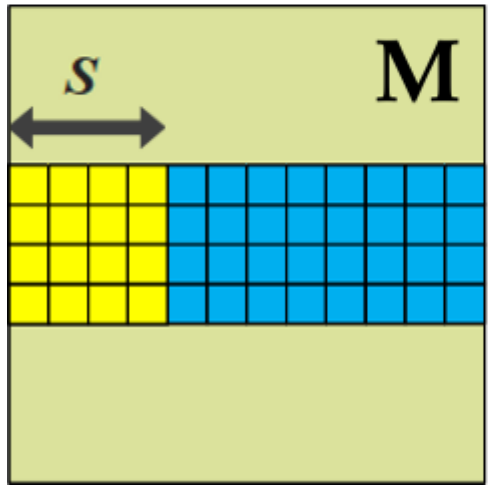

Benefits:

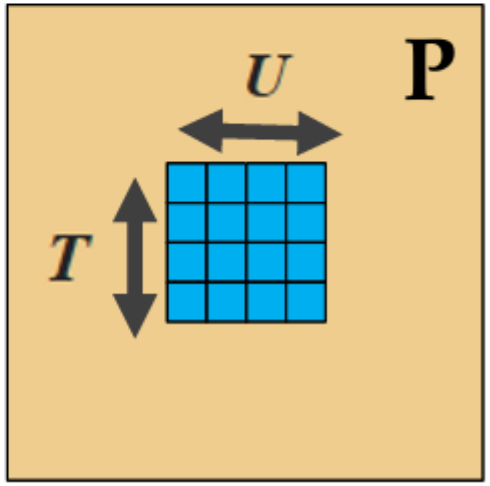
Tiling allows for **reusing** input elements

**U** times per element for M

**T** times per element for N

Reuse also happens in Shared Memory (fast)

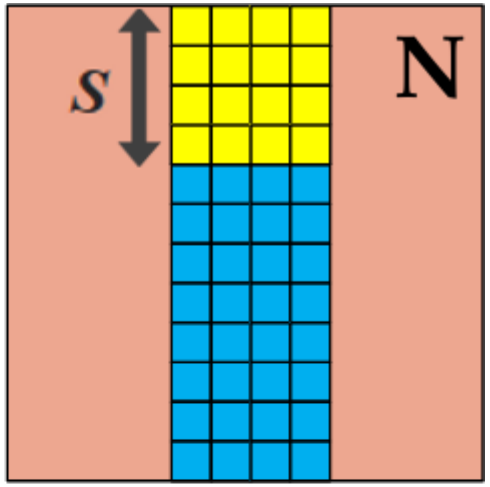


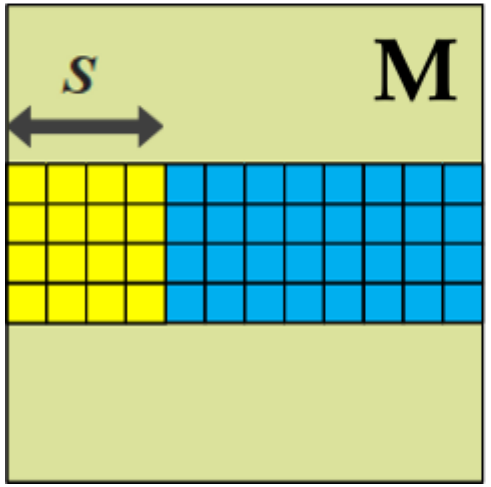



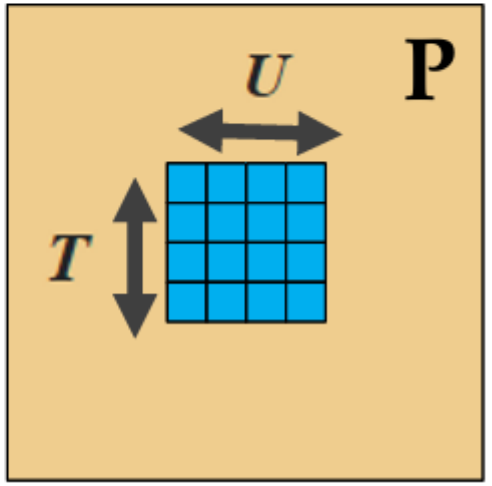
# Basic Shared Memory Tiling

Is a ***square*** thread block layout necessary? (**T** = **U**)

What about S? Does it need to match **T** and/or **U**?



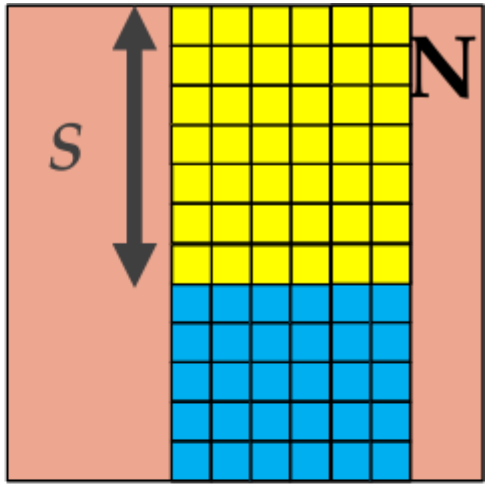



# Parameter Tuning in Basic Shared Memory Tiling

Thread block (output tile) can be non-square **(T x U)**

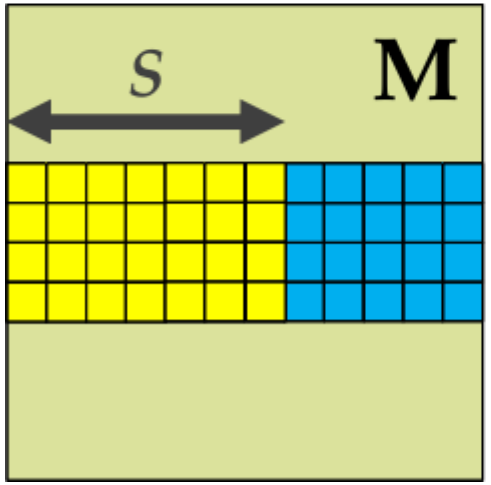
S (input tile shared dimension) can be flexible

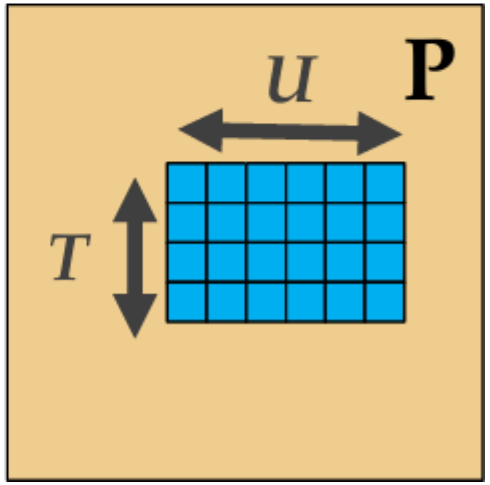
Each thread loads **(S x T) / (T x U)** elements from M and **(S x U) / (T x U)** from N.




But why?
Larger **T** and **U** allows for more reuse
*(recall they each represent reuse in M and N)*

Larger **S** allows for less __syncthreads()



# Basic Shared Memory Tiling Efficiency

Your block size (**U**x**T**) is limited, can't increase much (32 x 32 = 1024).

For the GPUs we use in delta (NVIDIA A40), biggest thread block is **32 x 32**

Each input loaded is used 32 times in shared memory

A40's peak FP32 **compute** throughput is *37.4 TFLOPs* (*37400 GFLOPs*), but peak **memory** throughput is only *696 GB/s*, so *32x* reuse means *5568* GFLOPs, far less than the peak obtainable compute throughput.

Basic shared memory tiling matmul **performs poorly** on modern GPUs without further optimizations.

## So What's Better than Shared Memory?

Registers!

Pros:
Registers have low access latency and high throughput

Basic shared memory tiling makes little use of the register hardware

However, registers are also local to each thread, meaning no reuse between different threads! (Con)

# Thread Coarsening Comes to the Rescue

In the Reduction Trees lecture (*Lecture 15*), thread coarsening was introduced to better utilize resources.

We can do the same here!

Instead of calculating **one** output value per thread, we coarsen and have **each thread** calculate **multiple** output values.

This will then allow us to utilize the **registers** in each thread to create more reuse!

New Matrix Multiplication Strategy: **Joint Register and Shared Memory Tiling**

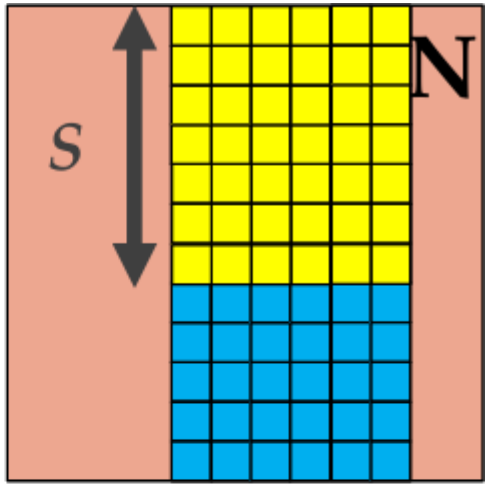
# Joint Register and Shared Memory Tiling

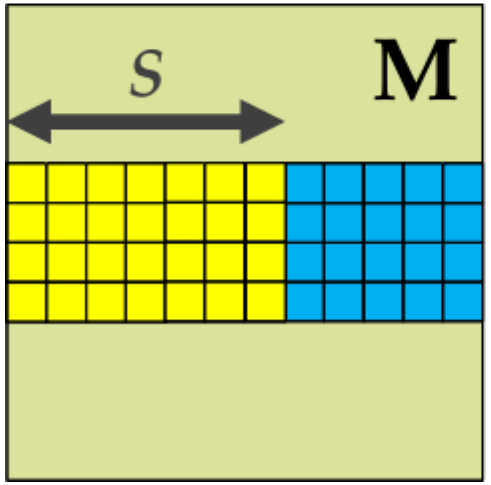
We make each thread block **1D**, with **T** threads per block.

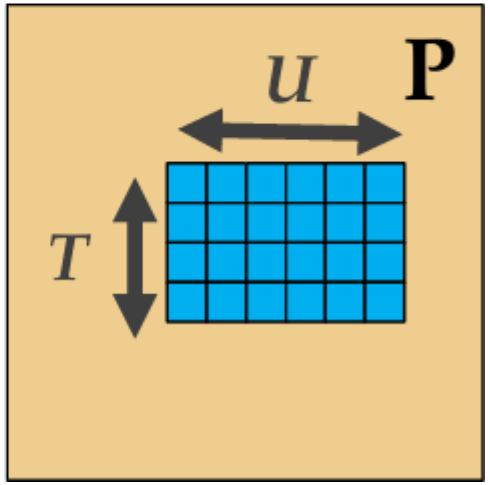
Thread coarsen, each **thread** computes **U** output elements

Each thread is then in charge of loading (**S x T) / T** elements from M and (**S x U) / T** elements from N (per tile)

*Remember, it's still up to you to decide how big **S** is.*



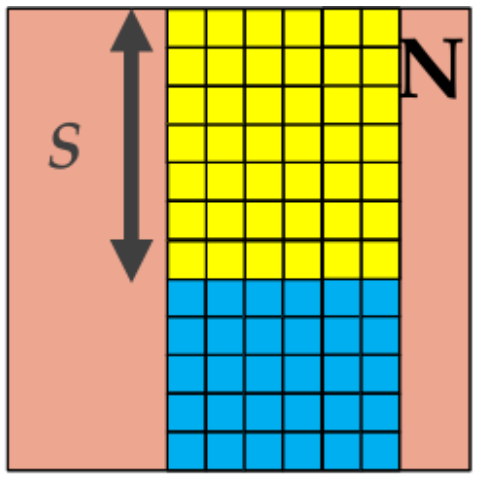



# Joint Register and Shared Memory Tiling

Either one of the 2 input tiles can be placed into registers.

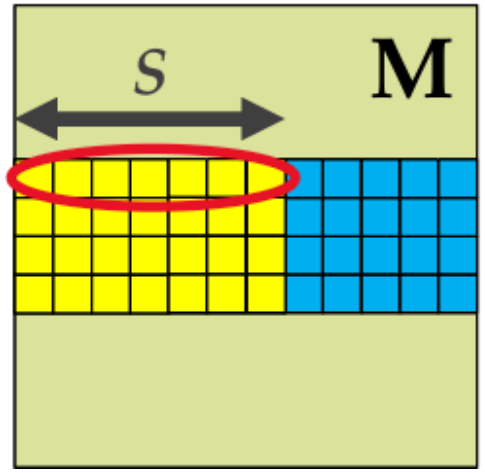
Let's just pick M (for no good reason, can switch based on data storage).

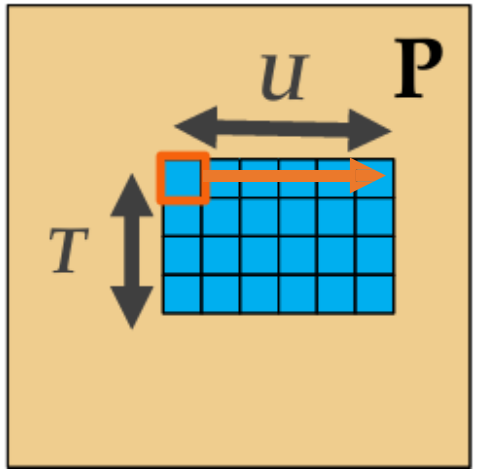



Each row of an M tile is loaded into a single thread's registers

A thread then uses the registered values to compute an entire row of **U** output values.

*Don't make U too big, you don't have an unlimited number of registers*
**U = 16** is a reasonable size.



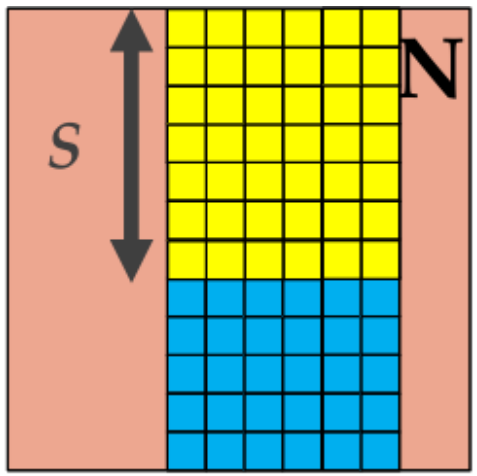



# Joint Register and Shared Memory Tiling

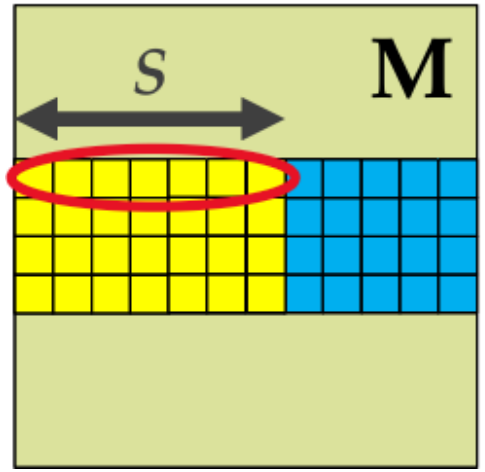
The output tile height **T** is now the number of threads in the block, and you'll probably want more than 4.

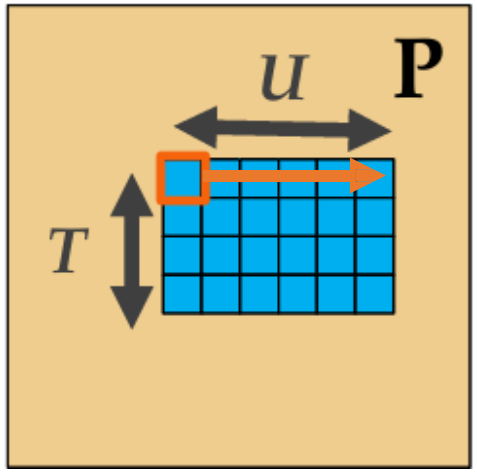
Remember the dimension of the output tile decides how much reuse you get.

But don't make it too big that you start to run out of registers, good to do some parameter sweeping.



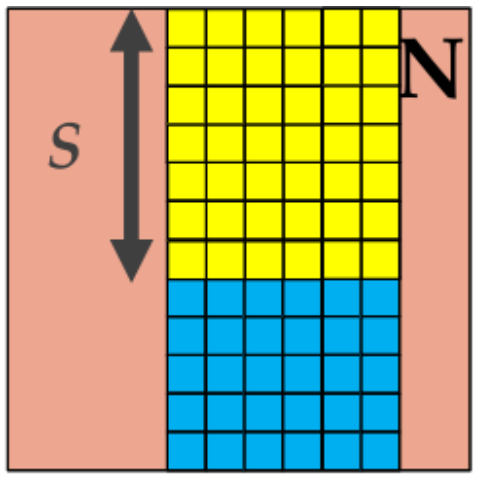

Any questions so far?



# Joint Register and Shared Memory Tiling

We also need to load the input tile from N into shared memory

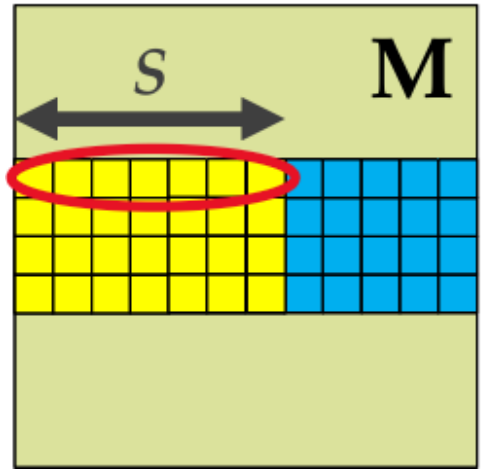


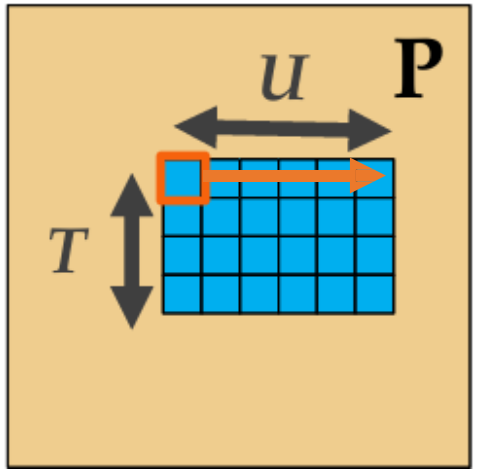

We don't want to load in multiple steps, so let each thread in a block load **1** value of N into a shared memory tile.

Given **U** and **T**, What should **S** be then?

Since we have **T** threads, and there are **S**x**U** elements in an N tile, each thread loading **1** value would mean **S** x **U** = **T**

**S = T / U**

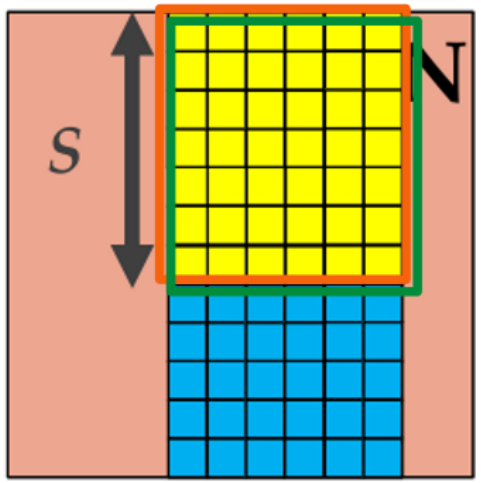



*You can also choose to load more than 1 input per thread, but it's more complicated and maybe slower.*

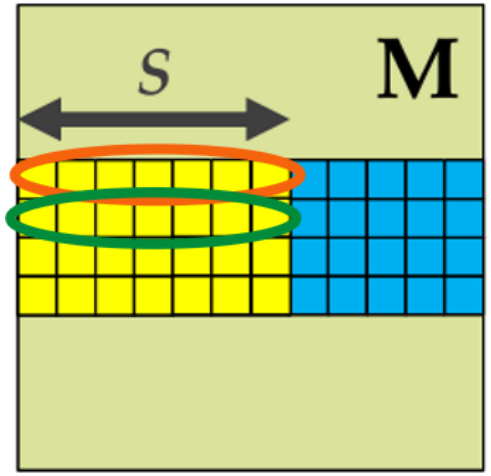
# Joint Register and Shared Memory Tiling

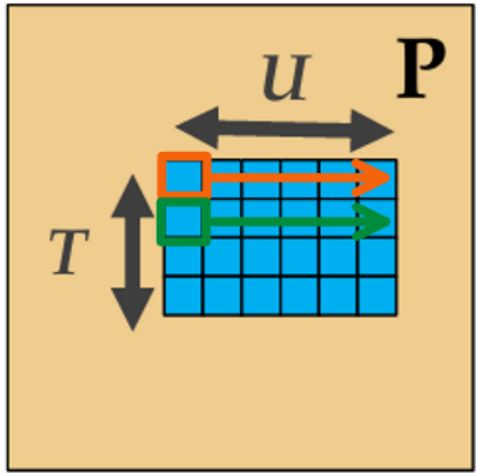
After a tile of M and a tile of N is loaded, the actual computation is simple.




Each thread uses the row of M in its registers and the entire tile of N in shared memory to multiply and accumulate to U different output values.

Then start over again, load the next tile of M and N, compute, add to the U different output values




You should probably have registers for the output values of each thread.

# Joint Register and Shared Memory Tiling

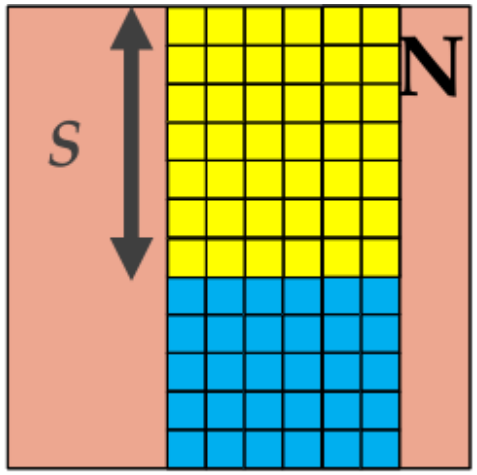
To review, in each iteration,

- Threads in a block collaboratively load an **S×U** tile of N into shared memory
- Each thread loads **S** elements from M into registers (together, they form a **T×S** tile of M). *You can have a thread load all **S** elements it needs at once, or just load one, compute, then load the next.*
- Each of **T** threads calculates and accumulates to the partial result of its **U** output values.
- Don't forget your __*syncthreads()*!

<u>You will need to implement this as an optimization in your final project.</u>

# Joint Register and Shared Memory Tiling

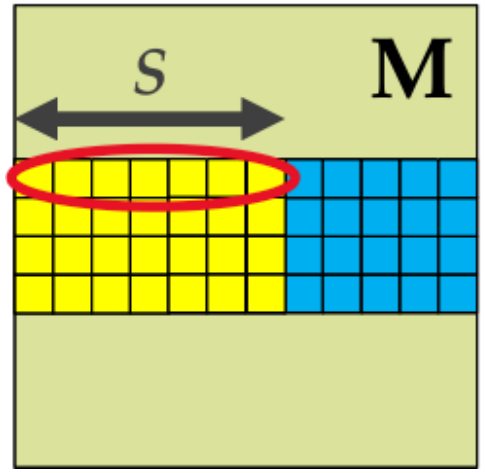
Something to think about:
Does this algorithm have coalesced loads for M and N?

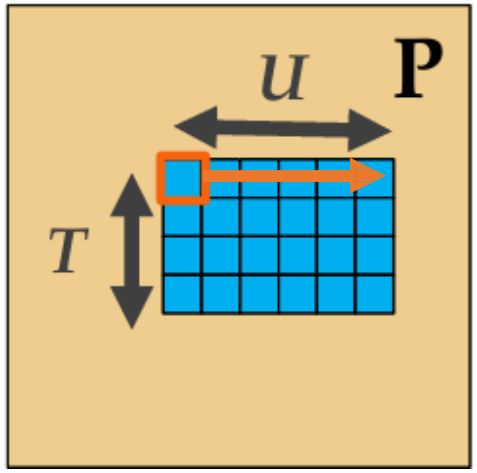



For loading N into shared memory, it's easy to make sure loads are coalesced.

But what about M?

Each thread loads a row of consecutive values from M…




Consecutive threads load from other rows…

Uh Oh…

# What About This?

Earlier, we used the value of 37.4 TFLOPs as the peak compute throughput of the A40 GPU.

But there is another important number:

74.8 TFLOPs of TF32 compute throughput

How?

**Tensor Cores!**

(The 2x ratio is coincidence)

| | |
|---|---|
| GPU architecture | NVIDIA Ampere architecture |
| GPU memory | 48 GB GDDR6 with ECC |
| Memory bandwidth | 696 GB/s |
| Interconnect interface | NVIDIA® NVLink® 112.5 GB/s (bidirectional)[3] PCIe Gen4: 64GB/s |
| NVIDIA Ampere architecture-based CUDA Cores | 10,752 |
| NVIDIA second-generation RT Cores | 84 |
| NVIDIA third-generation Tensor Cores | 336 |
| Peak FP32 TFLOPS (non-Tensor) | 37.4 |
| Peak FP16 Tensor TFLOPS with FP16 Accumulate | 149.7 \| 299.4* |
| Peak TF32 Tensor TFLOPS | 74.8 \| 149.6* |

Image Source: NVIDIA A40 GPU Data Sheet

# Tensor Cores

Tensor Cores are specialized matrix-matrix hardware compute units introduced in NVIDIA GPUs after their **Volta** generation.

This **hardware** addition significantly increases Matrix Multiplication efficiency.

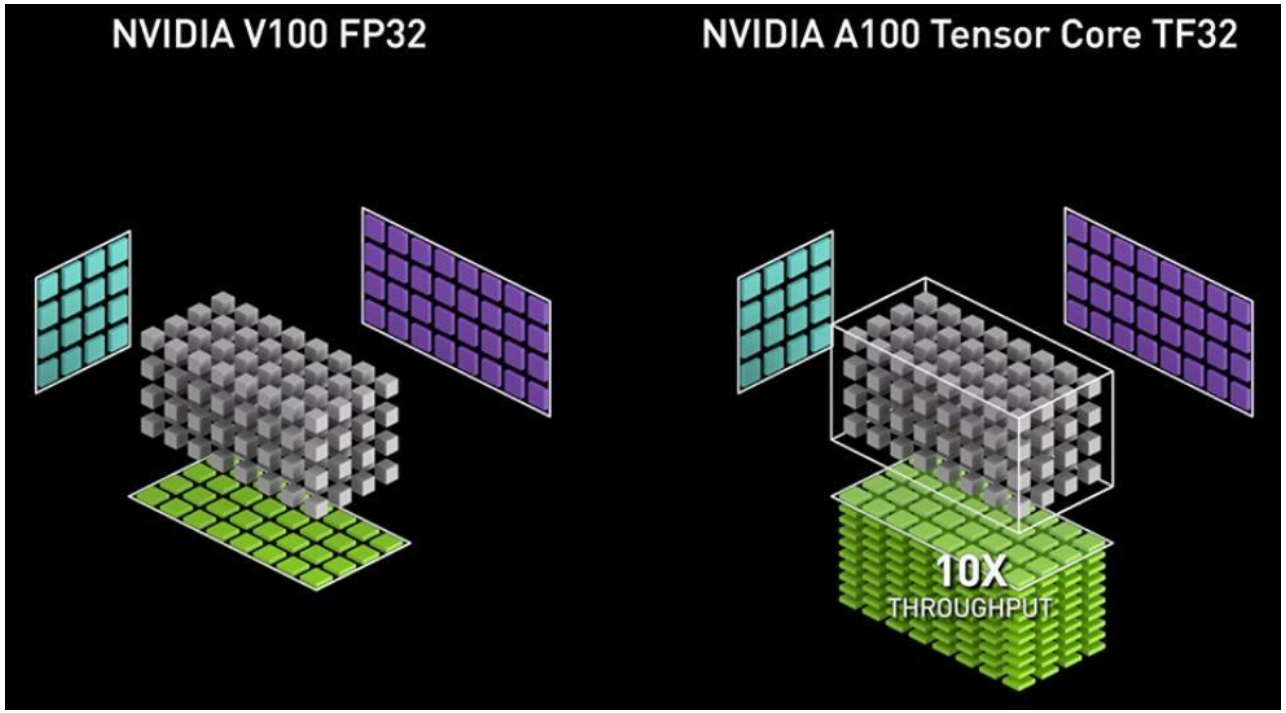



Image Source: NVIDIA Blog Post

# Tensor Cores – TF32?

**TF32** is a special **floating-point format** used in NVIDIA Tensor Cores

TF32 stands for Tensor Float [*Not Quite*] 32-bit format

Less precision than FP32, but good enough for Deep Learning

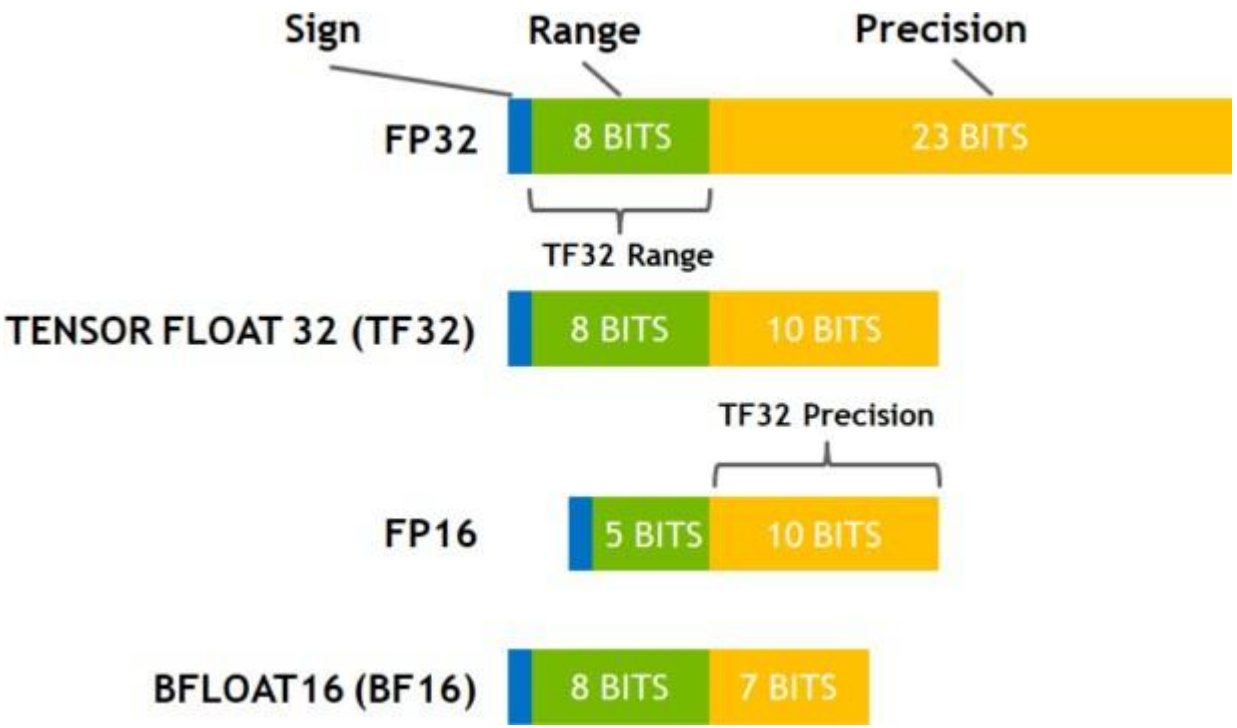



Image Source: NVIDIA Blog Post

# Tensor Cores – Using TF32

**TF32** shares the same exponent format as FP32, and the mantissa format is simply a less precise version of FP32's mantissa bits, so any TF32 value can also be stored in a float variable.

The cuda intrinsic "*__float_to_tf32*" can be used to cast a float down to TF32.

```
#include <mma.h>
using namespace nvcuda; // for use of nvcuda::wmma

__global__ void some_kernel(...) {
    float fp32_val = 3.141592653;
    float tf32_val = wmma::__float_to_tf32(fp32_val);

    printf("fp32_val: %.10g\n", fp32_val);
    printf("tf32_val: %.10g\n", tf32_val);
}
```

```
job starting on gpub001.delta.ncsa.illinois.edu

fp32_val: 3.141592741
tf32_val: 3.140625
```

Image Source: NVIDIA Blog Post

# The WMMA API

Under the *nvcuda* namespace, NVIDIA defines the *wmma* (**Warp** Matrix Multiply and Accumulate) API with the following functions (and more):

```
wmma::fragment<typename Use, int m, int n, int k, typename T, typename Layout=void> name;

void load_matrix_sync(fragment<...> &a, const T* ptr, unsigned ldm);

void load_matrix_sync(fragment<...> &a, const T* ptr, unsigned ldm, layout_t layout);

void store_matrix_sync(T* ptr, const fragment<...> &a, unsigned ldm, layout_t layout);

void fill_fragment(fragment<...> &a, const T& val);

void mma_sync(fragment<...> &d, const fragment<...> &a, const fragment<...> &b, const fragment<...> &c);

float __float_to_tf32(float val);
```

Source: Cuda C++ Programming Guide

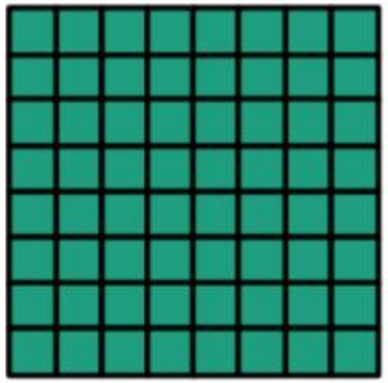
# Using WMMA to Accelerate Matrix Multiplication

## First, create the fragment that the tensor cores will use to access inputs.

- The first parameter can be ***matrix_a***, when the fragment is used as the first multiplicand; ***matrix_b***, when the fragment is used as the second multiplicand; or ***accumulator***, when the fragment is used as the source or destination accumulators.

- The ***m***, ***n*** and ***k*** sizes describe the shape of fragment. For *matrix_a*, the tile takes dimension m x k; for *matrix_b* the dimension is k x n, and *accumulator* tiles are m x n.

- The data type, ***T***, may be ***double, float, __half, __nv_bfloat16, char, unsigned char,*** *or* ***wmma::precision::tf32*** for multiplicands. For accumulators, it can be ***double, float, int,*** *or* ***__half***.



Per-Thread fragments to hold components of matrices for use with Tensor Cores

```
wmma::fragment<typename Use, int m, int n, int k, typename T, typename Layout=void> name;
```

# Using WMMA to Accelerate Matrix Multiplication

NVIDIA has certain restrictions on the fragment dimensions for tensor core:

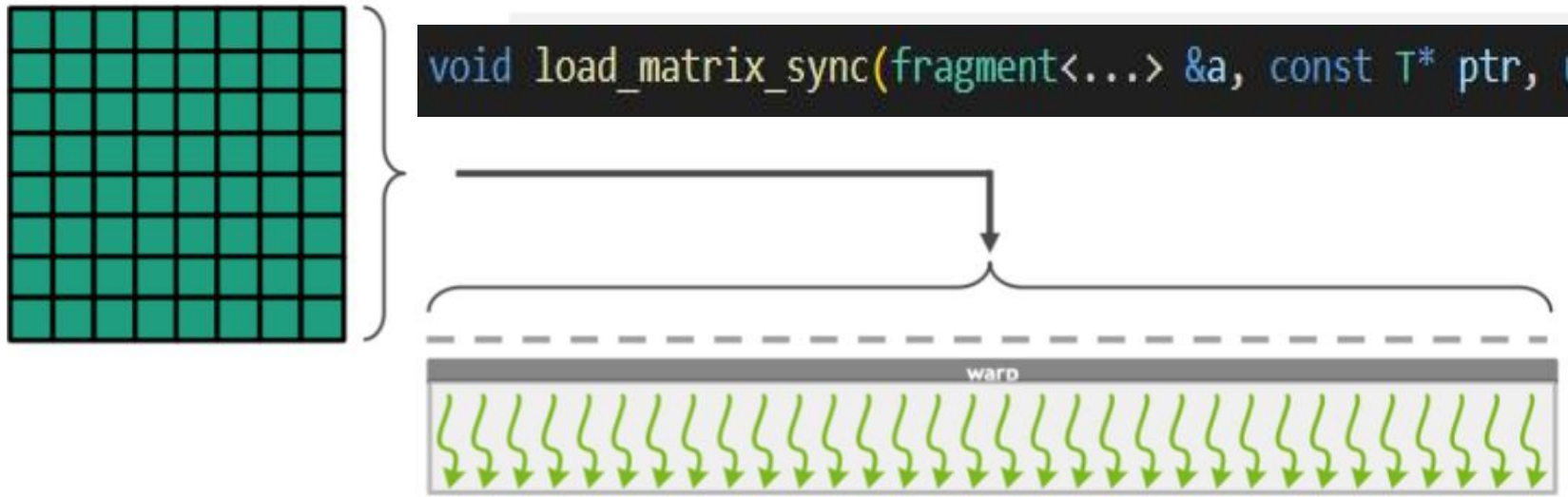
| Matrix A | Matrix B | Accumulator | Matrix Size (m-n-k) |
|---|---|---|---|
| __half | __half | float | 16x16x16 |
| __half | __half | float | 32x8x16 |
| __half | __half | float | 8x32x16 |
| __half | __half | __half | 16x16x16 |
| __half | __half | __half | 32x8x16 |
| ⋮ | ⋮ | ⋮ | ⋮ |
| __nv_bfloat16 | __nv_bfloat16 | float | 8x32x16 |
| precision::tf32 | precision::tf32 | float | 16x16x8 |

# Using WMMA to Accelerate Matrix Multiplication

Then data is loaded from memory (global, shared, register, etc.) into the tensor core fragments.

Note that the third parameter of *load_matrix_sync* is the leading dimension of the input in memory (number of columns for row-major).

Warp-level operation to fetch components of matrices into fragments




```
void load_matrix_sync(fragment<...> &a, const T* ptr, unsigned ldm, layout_t layout);
```

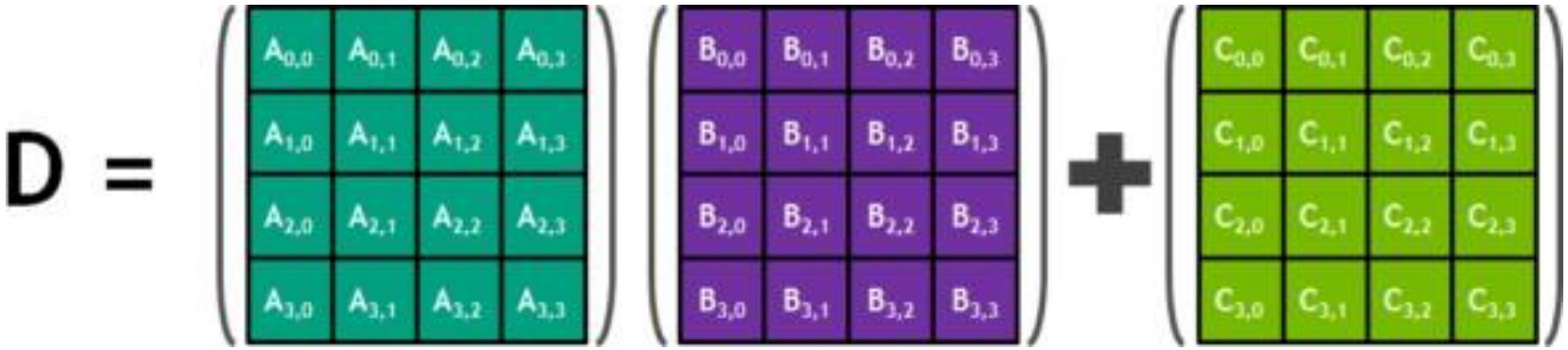
# Using WMMA to Accelerate Matrix Multiplication

Tensor Core units are composed of hardware matrix processing arrays that performs the operation **D** = **A** * **B** + **C** in a warp-synchronizing fashion.

Results are then distributed across warps.

Warp-level operation to perform matrix multiply and accumulate

```
void mma_sync(fragment<...> &d, const fragment<...> &a, const fragment<...> &b, const fragment<...> &c);
```

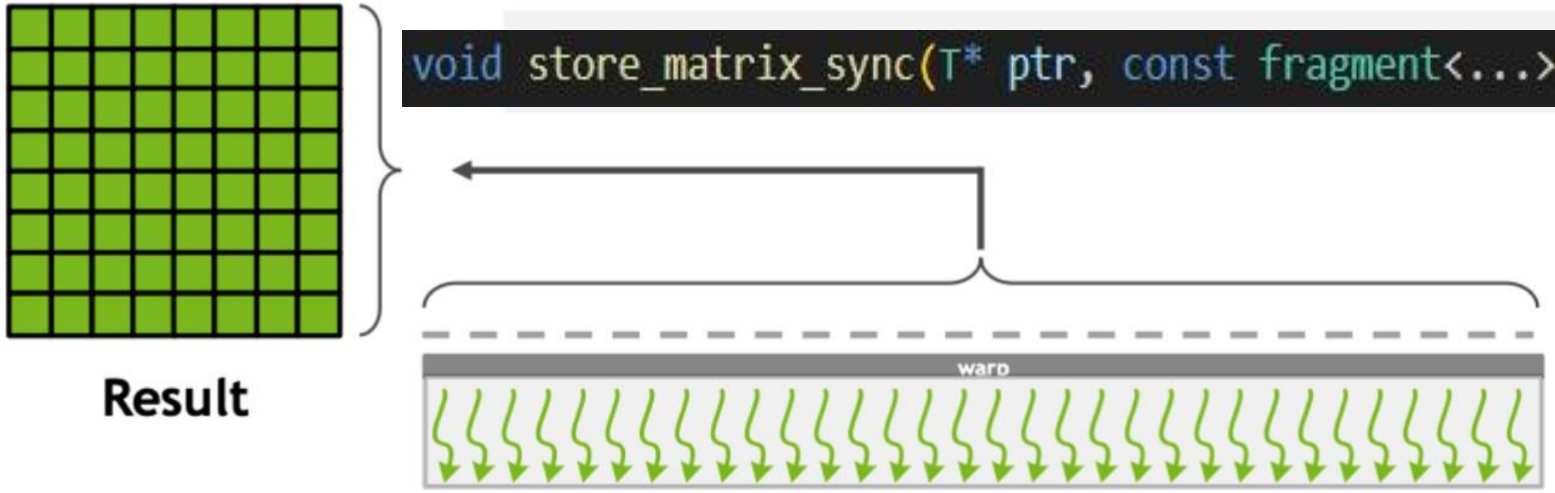


# Using WMMA to Accelerate Matrix Multiplication

Finally, the data can be loaded back to memory (shared, global, register, etc.) similar to the initial loading.

Warp-level operation to fetch components of matrices into fragments

```
void store_matrix_sync(T* ptr, const fragment<...> &a, unsigned ldm, layout_t layout);
```



# Using WMMA to Accelerate Matrix Multiplication

Full examples can be found at [this NVIDIA Blog Post](), and in the [CUDA C++ Programming Guide]().

## TENSOR CORE EXAMPLE

Create Fragments

Initialize Fragments

Perform MatMul

Store Results

```
__device__ void tensor_op_16_16_16(
    float *d, half *a, half *b, float *c)
{
  wmma::fragment<matrix_a, …> Amat;
  wmma::fragment<matrix_b, …> Bmat;
  wmma::fragment<matrix_c, …> Cmat;

  wmma::load_matrix_sync(Amat, a, 16);
  wmma::load_matrix_sync(Bmat, b, 16);
  wmma::fill_fragment(Cmat, 0.0f);

  wmma::mma_sync(Cmat, Amat, Bmat, Cmat);

  wmma::store_matrix_sync(d, Cmat, 16,
                          wmma::row_major);
}
```

**CUDA C++**

**Warp-Level Matrix Operations**

# cuBLAS – Can you beat the professionals?

Matrix multiplication is a huge bottleneck, spend most of your time here for machine learning workloads.

Model providers are spending hundreds of millions of dollars **per** model, just for training (almost $200 million for Gemini 1.0).

If you optimize matrix multiplication even by 1% for a specific model, you save millions of dollars.

NVIDIA makes the GPUs, and has a very good reason to tackle this problem themselves.

# cuBLAS – Can you beat the professionals? (continued)

Solution: Pay specialized teams of engineers 6+ figure salaries to write hundreds (maybe thousands) of variants of kernels to fit a variety of shapes, precisions, layouts, etc.

Replaces entire kernels in a single line. **(DO NOT USE cuBLAS UNLESS SPECIFIED FOR AN OPTIMIZATION).**

Use a heuristic model to determine which variant fits the user's needs the best.

Can you beat the people who make the GPUs?
Yes, often you can make something that fits your needs the best and beat cuBLAS.

# Skinny Matrices

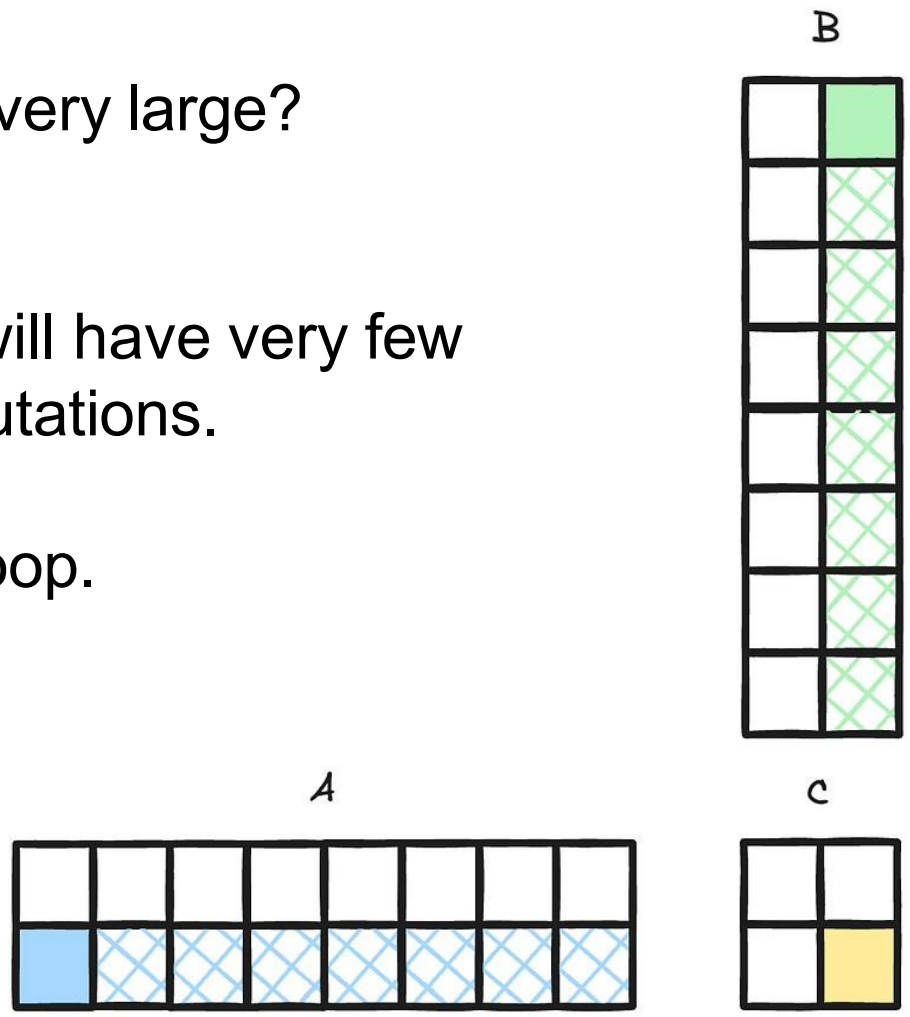
What happens when we have skinny matrices?

Specifically, matrices where the inner dimension is very large?

Using our existing thread block tiling mapping, we will have very few blocks launched to perform large amounts of computations.

There will be many iterations of the outer tiling for loop.

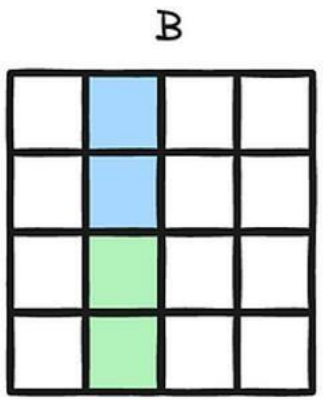
Big waste of time and resources!



# Split-K

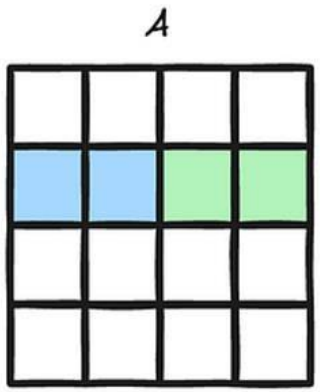
The Split-K algorithm for matrix multiplications parallelizes along the K dimension (shared dimension between two input matrices).

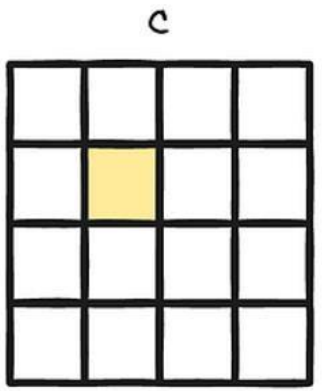
Here we split K into **2**, each thread block loads a half chunk of a row from input A and half of a column from input B for 1 block of output.




Note that here the 2 thread blocks are assigned to calculate the same block of output values!

After independently loading part of the input and computing partial results, thread blocks **atomically** add to the output, forming the final output values.



# Split-K

Some things to think about:

- How many splits you do along K is a tuning variable, sometimes a split of 1 is the best, other times you would need a lot of splits, depends on your inputs.
- The final atomic accumulation of all partial results may be costly if you directly add from each block to global memory. If you split K among many blocks, consider using a reduction kernel to sum all the partial results.
- The order of the final partial result accumulation will probably vary, since floating point math is not associative, think twice if this loss of numerical correctness is acceptable.

# Pointer Aliasing

Here's a straightforward snippet of code

```
__global__ void alias(float* a, float* b, float* c) {
  int index = blockIdx.x * blockDim.x + threadIdx.x;
  // assume no thread reads out of bounds of inputs
  a[index] = a[index] + c[index];
  b[index] = b[index] + c[index];
```

*Or is it?*

What if ***a*** and ***c*** *point to the same address*?

*Did you think about that when you saw the code?*

The **compiler** definitely did

# Pointer Aliasing

```
__global__ void alias(float* a, float* b, int* c) {
  int index = blockIdx.x * blockDim.x + threadIdx.x;
  // assume no thread reads out of bounds of inputs
  a[index] = a[index] + c[index];
  b[index] = b[index] + c[index];
  // ...
```

The compiler must conservatively assume pointers alias, so it will add code redundancy during compilation to ensure correctness (even if the programmer knows that the pointers never alias). This causes **inefficient** code!

In the example, instead of putting the initially loaded **c** value into a register, the compiler is forced to generate code that loads **c** from global memory twice!

# Compiler Optimizations – The Restrict Keyword

```
__global__ void alias(float* __restrict__ a, float* __restrict__ b, int* __restrict__ c) {
  int index = blockIdx.x * blockDim.x + threadIdx.x;
  // assume no thread reads out of bounds of inputs
  a[index] = a[index] + c[index];
  b[index] = b[index] + c[index];
  // ...
```

The "*__restrict__*" keyword promises the compiler that any data **written** through that pointer is **not read** by any other pointer <u>that also has the restrict property</u>.

With the help of the "*__restrict__*" keyword, the compiler will be able to aggressively optimize any use of the pointers (registering, reordering, etc.), which can bring significant performance improvements in certain use cases.

# For-Loop Overhead

```c
for (size_t i = 0; i < N; i++) {
  sum += A[i];
}
```

For Loop in C Code

```
LOOP
    BR to END if not (i < N)
    sum A[i];
    JMP to LOOP;
END
```

For Loop in Execution

If **N** is large, for-loop overheads will be significant in execution.

Say N is 10,000, then we will need to compute 10,000 loop conditions and loop increments as overheads for the actual task.

# Compiler Optimizations – Loop Unrolling

Loop unrolling is an optimization technique that reduces the number of iterations in for-loops by explicitly executing multiple steps into one iteration.

```
for (size_t i = 0; i < N; i = i + 2) {
  sum += A[i];
  sum += A[i + 1];
}
```

```
#pragma unroll 2
for (size_t i = 0; i < N; i++) {
  sum += A[i];
}
```

By either manually unrolling (left) or using a compiler directive for unrolling (right), you may be able to achieve noticeable performance improvement <u>after careful and thoughtful tuning</u>.

*Manually unrolling will also require code handling the case of N not being divisible by the unrolling parameter.*

# Be Careful!

By default, the Cuda compiler already performs Loop Unrolling aggressively, but there can still be room for improvement through manual tuning for specific use cases.

Unrolling reduces loop overheads (e.g., exit conditions, loop variable increments) but also increases code size.

Improper unrolling parameters much may not help (even make things slower) since there is a trade-off between total number of instructions and the loop overheads.

Unrolling characteristics will likely be different for each application and may also differ significantly within the same application based on input sizes.

# Global Memory Pressure in Self Attention

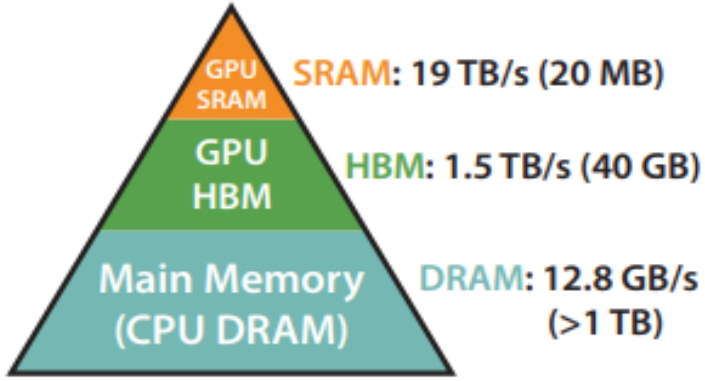
The standard self-attention formula defines the output "**O**" as:

$$S = QK^T, \quad P = \mathrm{softmax}(S), \quad O = PV$$

For a sequence length of "**N**," must read and write both matrices "**S**" and "**P**" to global mem., of size **N** x **N** respectively.

Each function as a separate kernel.

Earlier solutions almost entirely focused on FLOP reduction, rather than this IO bottleneck.




Memory Hierarchy with Bandwidth & Memory Size

# IO-Awareness and Tiling

FlashAttention computes exact mathematical result of standard attention, but restructures access patterns to fit into cache (shared memory).

Loads once: tiles of the Key and Value matrices from global mem. into shared mem.
Sweeps: The entire Query matrix tile-by-tile to compute the output

For a cache size of **M** elements, head dimension of **d**:
Cache can hold (**M/d**) tokens, call it **x**
Number of passes for length **N**, is **N/x**
Total global traffic is the number of passes by the size of Query matrix

$$O(N^2 d^2 / M)$$

# Questions?